

Robotique
CRC
Robotics

PROGRAMMING COMPETITION



2027

PRELIMINARY PROBLEM 1

A FEW NOTES

- The complete rules are in section 4 of the rulebook.
- You have until **Friday, November 13th, 11:59 pm** to submit your code.
- Feel free to use the programming forum on the CRC discord to ask questions and discuss the problem with other teams. It is there for that purpose!
- **We are giving you quick and easy-to-use template files for your code and the tests. You are required to use them.**

USING THE TEMPLATE FILE

- The template for each challenge contains where to write your code and the tests for the challenge. To run the tests you can click play ▸ in any editor like VSCode or run the file using python. There will also be Jupyter files available if you prefer using those instead.
- Points given in the document are indications of how difficult the section is and how many points you will get if you complete it. This preliminary problem is going to be 2% of the main challenge.

STRUCTURE

Every problem contains a small introduction like this about the basics of the problem and what is required to solve it.

Input and output specification:

Contains the inputs and their format, and which outputs the code is required to produce and in what format they shall be.

Sample input and output:

Contains a sample input, sometimes containing sub examples in the sample input, and what your program should return as an output.

Explanation of the first output:

Explains briefly the logic that was used to reach the first output, given the first input.

Cookies!

Nothing is more lovely than some cookies to start the day — they are so delicious that they can make a boring Monday 5 AM feel like a celebration. There is so much that can be said about those tasty circles, so let's go ahead and dive in some of our favorite topics about cookies! 🍪

Part 1: Cookies online (10 points)

Cookies are amazing when we talk about food, but they are also a really important concept on the internet. Cookies are created on almost every website to help you not have to connect again every time or to remember one of your preferences. They have so many uses, but today we'll focus on finding and creating cookies.

We are going to create a personalized cookie for every user that is requesting one and does not have one already. If the user has a cookie, we will give their cookie back, otherwise we will create a new one for them. To create a unique cookie we will use the first 5 letters of the username followed by the random text given to you. All first five letter of the username will be unique for each user.

Input and output specification:

For input, you will receive :

cookies : The list of already existing cookies
username: The username of the person trying to get a cookie
random_text: some random text to use if you want to create a new cookie

As output you will need to give back the cookie of the user from the cookies list if there is one that matches and otherwise give back the cookie you just created for them.

Sample input:

```
cookies = [“frankRCdsl3csW8”, “rando3J5a9icd0s”]
username = biggest_crc_fan
random_text = 4mvdo1fe2nf833c

cookies = [“frankRCdsl3csW8”, “rando3J5a9icd0s”]
username = frank_crc
random_text = 4mvdo1fe2nf833c

cookies = [“frankRCdsl3csW8”, “rando3J5a9icd0s”]
username = random_test
random_text = 4mvdo1fe2nf833c
```

Sample output:

```
bigge4mvdo1fe2nf833c
frankRCdsl3csW8
rando3J5a9icd0s
```

Explanation of the first output:

In the first example we take the first 5 letters of the username and get “bigge”. We look at the list of cookies that are provided to us. Since none of them start with the name of our user, it means that we need to create them a new cookie. We combine the first 5 letters with the random text to have our final cookie.

In the second example, we take the first 5 letters of the username and get “frank”. We look in the list of cookies and there is one that starts with frank. We take that cookie from the list and return it to the user since they already have a valid cookie!

Part 2: Cookie sales (10 points)

We want to organize a cookie sale to get more budget towards the upcoming competition. We want to know how many cookies we need to sell to start to make profit and how much final profit we get at the end of the day.

You need to determine how many cookies need to be sold to start getting profit and then the rest of the profit for the day. All the cookies for a day will be the same price to facilitate the calculations. There will always be enough cookies for profit — you will never be at a loss.

Input and output specification:

For input, you will receive :
c : The cost to make the cookies *int*
p : the price that the cookies are sold at *float*
q : the quantity of cookies prepped for the day *int*

For output, you need to output a *list* of *int* the first one being the amount of cookies needed to be sold to get profit and the second one is the amount of profit in dollars for the day.

Sample input:

```
c = 20 p = 1.0 q = 160
c = 51 p = 1.50 q = 150
c = 35 p = 2.50 q = 68
c = 50 p = 0.50 q = 102
```

Sample output:

```
[20, 140]
[34, 174]
[14, 135]
[100, 1]
```

Explanation of the first output:

There are 20$ that were used to make the cookies and they are sold at 1$ each which would make 20 cookies sold to get profit and then there are 140 cookies left at 1$ each which makes 140$.

Part 3: Cookie recipe (30 points)

You bought a recipe book containing all the ingredients and steps to bake a single cookie. You have family members coming over for a halloween party and you would like to make enough cookies for everyone. You would like to have the necessary ingredients for any amount of cookies you would like to bake. Here is what you want to do:

- Multiply the required ingredients with the numbers of cookies to bake, with the oven temperature and the baking time as the exceptions; see below.
- Do not modify any *text* in the provided input instructions; you must only modify the quantities in the said instructions.
- Do not increase the temperature of the oven, you don't want to burn your house down. There will always be the word “oven” in the instruction that has the temperature.
- For the time, add 1 minute to the timer for every cookie starting from the 6th one. The instruction containing the time will always have the word “minutes”.

Input and output specification:

For input, you will receive a *string list* of ingredients/steps on baking a cookie and the amount of cookies you would like to bake.

For output, you will give the updated version of the list of ingredients/steps multiplied by the amount of cookies to bake.

Sample input:

```
num_of_cookies = 6
recipe = [
   “Preheat the oven to 375 degrees fahrenheit”,
   “Add 80 grams of flour into the bowl”,
   ”Add in 40 grams of brown sugar”,
   “Incorporate 30 grams of softened butter into the bowl”,
   “Blend in 2 eggs”,
   ”Mix the ingredients until the mixture is smooth”,
   “Pour the mixture into a tray then bake for 12 minutes”
]
```

Sample output:

```
[“Preheat the oven to 375 degrees fahrenheit”,
“Add 480 grams of flour into the bowl”,
”Add in 240 grams of brown sugar”,
“Incorporate 180 grams of softened butter into the bowl”,
“Blend in 12 eggs”,
”Mix the ingredients until the mixture is smooth”,
```

```
“Pour the mixture into a tray then bake for 13 minutes”]
```

Explanation of the first output:

In the first example, the number of grams of flour, brown sugar and softened butter were multiplied by the number of cookies (multiplied by 6). The value for the temperature stays unchanged.

We will also add 1 minute in the 12 minutes timer since we have at least 6 cookies (exactly 6, for 7 cookies it would be 2 minutes added). Which gives us a bake time of 13 minutes.

Part 4: Cookie factory (30 points)

Pastry Chef Biskwi had to deliver a flawless cookie order to the boss of the multinational corporation Choco & Co. The boss has an unhealthy obsession with everything being EXACTLY in order at all times, so Chef Biskwi invested in a machine that sorts the cookie boxes into ascending order by number right before each order ships.

But today, the worst has happened. The machine broke down at the exact moment the order was due. **IT'S A DISASTER!** Luckily, before it takes its dying breath, the machine can perform one last, single operation.

Here's how this miracle operation works: on the conveyor belt, there are n boxes, each with a different number between 1 and n (every number appears exactly once, with no duplicates and no missing numbers). The arm can:

- pick any group of positions on the belt (they don't have to be next to each other: the arm can grab the 2nd box, skip the 3rd, then grab the 4th, and so on);
- then put the boxes of that group back down, but in reverse order. In other words, the leftmost box of the group and the rightmost box of the group swap places, the 2nd from the left swaps with the 2nd from the right, and so on until the middle.

Any boxes that weren't picked stay exactly where they are.

Chef Biskwi needs to know, before even trying (the arm won't get a second chance, and any box put in the wrong spot goes straight into the trash...), whether a single operation is enough to put the whole belt in ascending order (1, 2, 3, ..., n).

Input and output specification:

The input line contains n numbers: the current (chaotic) order of the boxes on the belt. These n numbers are exactly the numbers from 1 to n, each appearing once, but all jumbled up.

For each order, output "YES" if Chef Biskwi can save the delivery with a single operation, or "NO" if the order is hopelessly lost (and Choco & Co. will have to wait... or worse, cancel the contract).

Sample inputs:

[1]
[4, 2, 3, 1]
[3, 4, 1, 2]
[2, 1, 3, 5, 4]
[1, 6, 3, 4, 5, 2]
[1, 2, 3]
[2, 1]
[3, 2, 1]
[2, 3, 1]
[5, 2, 3, 4, 1]
[2, 1, 4, 3]

Sample outputs:

“YES”
“YES”
“NO”
“NO”
“YES”
"YES"
"YES"
"YES"
"NO"
"YES"
"NO"

Explanation of the last output**:**

The belt shows boxes 1, 6, 3, 4, 5, 2 in that order. The arm grabs the boxes at positions 2 and 6 (the boxes numbered 6 and 2) and puts them back in reverse order in those same positions: the box at position 2 and the box at position 6 swap places. The belt now reads 1, 2, 3, 4, 5, 6. **It’s a miracle!** The order can ship on time.

Part 5: Be chocolate or not to be, that is the question (35 points)

Cookie dough is spread out on the counter as a list. Just like we don't place each chocolate chip on the dough one by one, but instead mix them into the dough at random, the chocolate chips, represented by the letter C, are placed randomly in the dough.

Once the dough is prepared and spread out, all that's left to do is cut out the cookies with a cookie cutter and put them in the oven. And voilà, cookies!

The unfortunate thing about this method is that you might end up with cookies without any chocolate chips. The horror!!!

Input and output specifications:

You will receive a *list* or *string* that will represent a cookie dough. That dough contains P for the dough part and C for the chocolate chips.

You will also receive an *int* that represents the size of the cookie cutter to cut the dough with. You have to make sure that all the dough is used and none is left behind.

As the output you will have to provide the amount of cookies that do not contain chocolate chips.

Input examples:

```
size = 2
[['P' 'P' 'P' 'P']
 ['P' 'C' 'C' 'P']
 ['C' 'P' 'C' 'P']
 ['P' 'C' 'C' 'C']]

size = 2
[['P' 'C' 'P' 'C' 'C' 'P' 'P' 'C']
 ['P' 'P' 'C' 'C' 'P' 'P' 'C' 'P']
 ['C' 'C' 'P' 'P' 'P' 'P' 'P' 'P']
 ['P' 'P' 'P' 'P' 'P' 'P' 'C' 'C']
 ['P' 'P' 'P' 'C' 'P' 'P' 'C' 'C']
 ['C' 'P' 'P' 'P' 'C' 'P' 'P' 'P']
 ['C' 'C' 'P' 'P' 'P' 'C' 'P' 'P']
 ['P' 'P' 'C' 'P' 'P' 'C' 'P' 'C']]
```

Output examples:

```
0
2
```

Explanation of the first output:

In the first example, we have a 4x4 grid and a cookie cutter of dimension 2 (so a 2x2). The first cookie is the top left 2x2 square:

P P

P C

We have at least one chocolate chip so this cookie is alright! For the next one top right we have:

P P

C P

There is also one chocolate chip so the cookie is ok! For the last 2 cookies (the bottom left and bottom right) we have one or more chocolate chips, so all the cookies are good and we have 0 cookies that do not contain any chocolate chip!

Part 6: Baking sheet (40 points)

Dum-E, Tony Stark's physical robotic workshop arm, is learning to bake cookies. Seeing it struggle to arrange the unbaked cookies onto the baking sheet, you decide to step in and write a short code to help the robotic arm place them.

Input and output specification:

For the input, you will receive a **list** of cookie **positions** and **sizes** (`[(x1, y1, radius1), (x2, y2, radius2), ...]`). For the output, you need to provide a **boolean** telling whether the arrangement is **valid or not** - valid being when there are no cookies overlapping nor touching.

Sample inputs:

1. `[(2, 3, 3)]`
2. `[(2, 3, 3), (5, 7, 1)]`
3. `[(2, 3, 3), (5, 7, 1), (2, 8, 2)]`
4. `[(2, 3, 3), (5, 7, 1), (9, 10, 5)]`

Sample outputs:

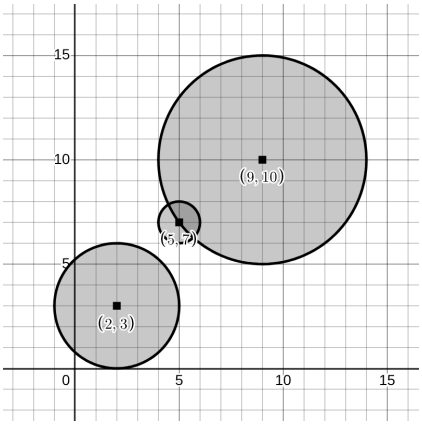
1. `True` - the only cookie on the baking sheet
2. `True` - there is an **empty space** of a distance of 1 between the two cookies
3. `False` - the first and third cookies are **pressed together** (borders touching)
4. `False` - the second and third cookies are **overlapping** (on top of each other)

Explanation of the fourth output:

In order to check the distance between two cookies, we can use the distance formula:

$$distance = \sqrt{(x1 - x2)^2 + (y1 - y2)^2}$$

Using the above formula, we get $\sqrt{(5 - 9)^2 + (7 - 10)^2} = 5$ units as the distance. However, the sum of their radiuses is already $1 + 5 = 6$ units, meaning the cookies are overlapping by 1 unit.



By using the same reasoning for the other cookie combinations, we can see that the others are not overlapping. But since **at least one** is overlapping, the whole layout is invalid and is `False` returned.